

Cost-Aware LLM Model Routing Using Optimization

An Optimization-Based Approach for Cost, Quality, and Latency-Aware
LLM Selection

Project Report

August 15, 2026

Contents

1	Project Overview	5
2	Problem Statement	5
3	Main Goal	6
4	Project Objectives	6
4.1	Minimize Cost	6
4.2	Maintain Response Quality	6
4.3	Reduce Latency	6
4.4	Optimize the Trade-off	6
5	Core Project Idea	6
5.1	Stage 1: Capability Prediction	7
5.2	Stage 2: Optimization	7
6	Dataset	7
6.1	Dataset Selected	7
6.2	Dataset Source	7
7	Selected LLMs	8
8	Dataset Structure	9

9	Complete Project Procedure	9
9.1	Step 1: Download the Dataset	9
9.2	Step 2: Load the Dataset	10
9.3	Step 3: Combine Model Results	10
9.4	Step 4: Data Cleaning	10
9.5	Step 5: Statistical Analysis	10
9.5.1	Average Quality	11
9.5.2	Total Cost	11
9.5.3	Average Cost	11
9.5.4	Token Usage	11
9.5.5	Execution Time	11
10	Query Complexity and Required Capability	11
11	Required Capability Levels	12
12	Query Feature Extraction	12
13	Capability Prediction	13
14	Optimization Technique 1: Integer Programming	13
14.1	Constraint 1: One Model Per Query	13
14.2	Constraint 2: Minimum Quality	14
14.3	Objective Function	14
15	Why Integer Programming?	14
16	Optimization Technique 2: Goal Programming	14
16.1	Goal Programming Objective	15
17	Optional Technique: AHP	15
18	Optimal Routing	16
19	Baseline Methods	16
19.1	Baseline 1: Cheapest Model	17
19.2	Baseline 2: Strongest Model	17
19.3	Proposed Method	17
20	Evaluation	17
20.1	Total Cost	17
20.2	Average Quality	17
20.3	Average Cost	17

20.4 Latency	17
21 Cost Saving	18
22 Quality Comparison	18
23 Quality-Cost Trade-off	18
24 Sensitivity Analysis	19
24.1 Minimum Quality Requirement	19
24.2 Cost Priority	19
24.3 Quality Priority	19
24.4 Latency Priority	19
25 User Interface	19
25.1 Dataset Upload	20
25.2 Statistics Dashboard	20
25.3 Optimization Settings	20
25.4 Results Dashboard	21
26 System Architecture	22
27 Technology Stack	22
27.1 Programming Language	22
27.2 Data Processing	22
27.3 Machine Learning	22
27.4 Optimization	22
27.5 Visualization	23
27.6 User Interface	23
28 Project Folder Structure	23
29 Complete End-to-End Procedure	24
30 Expected Outcome	25
31 Proof of Optimization Effectiveness	26
32 Limitations	26
33 Future Improvements	27
34 Final Project Summary	27

35 Final Project Flow	28
36 Central Concept	28

1 Project Overview

Large Language Models (LLMs) such as GPT, Claude, Gemini, and DeepSeek provide different levels of response quality, cost, and response time. Using a powerful and expensive model for every query is not always necessary. Simple queries may be handled effectively by smaller and cheaper models, while complex reasoning queries may require more capable models.

Using one powerful model for every query can therefore result in unnecessary computational cost and increased latency.

This project proposes an optimization-based LLM routing system that selects a suitable LLM for each query while considering response quality, cost, and latency.

The system consists of two major stages:

1. Predict the capability required by the query.
2. Use optimization techniques to select the most suitable LLM.

The primary optimization techniques used in this project are:

- Integer Programming
- Goal Programming

Analytic Hierarchy Process (AHP) may optionally be used as a supporting technique for model ranking.

2 Problem Statement

Different user queries require different levels of LLM capability. Sending every query to the most powerful model can provide high response quality, but it can also result in unnecessary cost and higher latency.

On the other hand, always selecting the cheapest model may reduce cost but can result in poor responses for difficult queries.

Therefore, the problem addressed in this project is:

How can we select the most suitable LLM for each query while minimizing cost and latency and maintaining the required response quality?

3 Main Goal

The main goal of this project is:

To develop an optimization-based LLM routing system that dynamically assigns each query to an appropriate language model while achieving a good balance between cost, response quality, and latency.

4 Project Objectives

4.1 Minimize Cost

Select a lower-cost LLM whenever it can provide the required quality.

4.2 Maintain Response Quality

Ensure that the selected LLM satisfies a predefined minimum quality requirement.

4.3 Reduce Latency

Prefer faster models whenever they provide sufficient quality.

4.4 Optimize the Trade-off

Find the best balance between:

- Cost
- Response Quality
- Latency

The objective is not simply to choose the cheapest or the most powerful model. The objective is to find the most suitable model for each query.

5 Core Project Idea

The proposed system consists of two main decision stages.

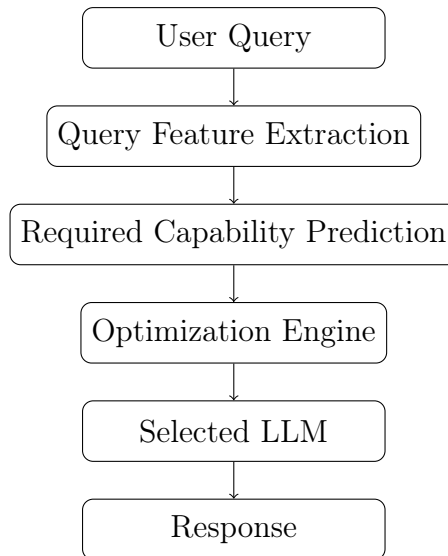


Figure 1: Basic project workflow

5.1 Stage 1: Capability Prediction

The first stage determines:

What level of LLM capability does this query require?

5.2 Stage 2: Optimization

The second stage determines:

Which available model can satisfy the requirement at the best cost-quality-latency trade-off?

6 Dataset

6.1 Dataset Selected

MMLU-Pro – Performance-Cost Setting

6.2 Dataset Source

LLMRouterBench

Official repository:

<https://github.com/ynulihao/LLMRouterBench>

The project uses the MMLU-Pro `test_3000` benchmark.

The dataset contains 3,000 questions and results from multiple LLMs.

The model result files contain information such as:

- Model name
- Dataset name
- Performance
- Execution time
- Prompt tokens
- Completion tokens
- Cost

Individual records contain:

- Query ID
- Original query
- Prompt
- Prompt tokens
- Completion tokens
- Cost
- Score
- Prediction
- Ground truth

7 Selected LLMs

A manageable subset of models will be selected from the available LLM results.

The recommended initial models are:

1. Gemini 2.5 Flash
2. DeepSeek V3
3. Gemini 2.5 Pro
4. Claude Sonnet 4
5. GPT-5

The selected models provide a range of cost and capability levels, which is necessary for meaningful model routing.

8 Dataset Structure

The original data contains model-level information similar to:

```
{
  "performance": 0.8093,
  "time_taken": 725.466,
  "prompt_tokens": 728048,
  "completion_tokens": 4296614,
  "cost": 10.9599,
  "counts": 3000,
  "model_name": "gemini-2.5-flash",
  "dataset_name": "mmlupro",
  "split": "test_3000",
  "records": [...]
}
```

Individual records contain information similar to:

```
{
  "index": 1,
  "origin_query": "...",
  "prompt_tokens": 236,
  "completion_tokens": 246,
  "cost": 0.0006858,
  "score": 1.0,
  "prediction": "B",
  "ground_truth": "B"
}
```

9 Complete Project Procedure

9.1 Step 1: Download the Dataset

Download LLMRouterBench and select:

```
results/
  bench/
    mmlu_pro/
      test_3000/
```

The required LLM result files are then selected.

9.2 Step 2: Load the Dataset

The JSON files for each model are loaded using Python.

The individual records are extracted from each model file.

9.3 Step 3: Combine Model Results

The results of all selected LLMs are combined into one standardized dataset.

The working dataset will have a structure similar to:

Table 1: Combined dataset structure

Query ID	Query	Model	Score	Cost	Prompt Tokens	Completion Tokens
1	Q1	Gemini Flash	1.0	0.001	236	246
1	Q1	DeepSeek V3	1.0	0.003	236	250
1	Q1	Gemini Pro	1.0	0.006	236	260
1	Q1	Claude	1.0	0.007	236	270
1	Q1	GPT-5	1.0	0.010	236	280

The numerical values shown above are illustrative. Actual values will be obtained from the dataset.

9.4 Step 4: Data Cleaning

The combined dataset will be checked for:

- Missing values
- Duplicate records
- Incorrect query alignment
- Invalid scores
- Invalid or negative costs
- Missing model names
- Missing token values

The same Query ID must refer to the same question across the selected models.

9.5 Step 5: Statistical Analysis

Before optimization, statistical analysis will be performed.

For each model, calculate:

9.5.1 Average Quality

$$\text{Average Quality} = \frac{\sum \text{Score}}{N}$$

9.5.2 Total Cost

$$\text{Total Cost} = \sum \text{Cost}$$

9.5.3 Average Cost

$$\text{Average Cost} = \frac{\text{Total Cost}}{N}$$

9.5.4 Token Usage

Calculate:

- Average prompt tokens
- Average completion tokens
- Total tokens

9.5.5 Execution Time

Execution time will be analyzed where reliable timing information is available.

10 Query Complexity and Required Capability

One of the main challenges in LLM routing is determining which model is appropriate for a given query.

The system should not simply assume:

$$\text{Long Prompt} = \text{Complex Query}$$

Instead, the actual performance of different LLMs on the same query is used to determine the required model capability.

For example:

Table 2: Example of capability determination

Query	Flash	DeepSeek	Gemini Pro	Claude	GPT-5
Q1	1	1	1	1	1
Q2	0	1	1	1	1
Q3	0	0	1	1	1

Assuming a minimum required quality of 90%:

- Q1 can be handled by a low-cost model.
- Q2 requires at least a medium-capability model.
- Q3 requires a higher-capability model.

11 Required Capability Levels

Capability levels are determined using the cheapest model that satisfies the required quality.

An example classification is:

Table 3: Example capability levels

Capability Level	Example Model
Level 1	Gemini 2.5 Flash
Level 2	DeepSeek V3
Level 3	Gemini 2.5 Pro / Claude Sonnet 4
Level 4	GPT-5

The actual capability levels will be derived from experimental results.

12 Query Feature Extraction

For a new query, the actual performance of every model is not known.

Therefore, features are extracted from the query.

Possible features include:

- Number of words
- Number of tokens
- Subject or domain
- Question type
- Numerical content
- Reasoning indicators
- Number of conditions

- Question structure

The goal is to learn:

$$\text{Query Features} \rightarrow \text{Required Capability}$$

13 Capability Prediction

A lightweight machine learning classifier can be used to predict the required capability level of a new query.

The process is:

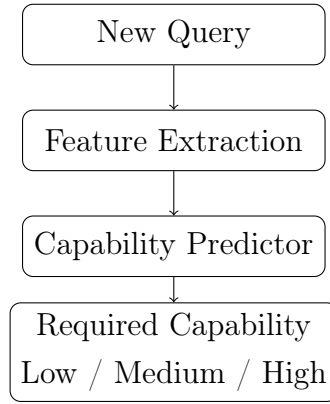


Figure 2: Capability prediction

The capability predictor does not perform the final model selection. It only estimates what level of model capability is required.

14 Optimization Technique 1: Integer Programming

Integer Programming is the primary optimization technique.

For query i and model j , define the binary decision variable:

$$x_{ij} = \begin{cases} 1, & \text{if query } i \text{ is assigned to model } j \\ 0, & \text{otherwise} \end{cases}$$

14.1 Constraint 1: One Model Per Query

Every query must be assigned to exactly one model:

$$\sum_j x_{ij} = 1$$

14.2 Constraint 2: Minimum Quality

The selected model must satisfy the required quality:

$$\sum_j Q_{ij}x_{ij} \geq Q_{min}$$

where:

$$Q_{min} = \text{minimum acceptable quality}$$

For example:

$$Q_{min} = 0.90$$

14.3 Objective Function

The main objective is to minimize total cost:

$$\boxed{\text{Minimize } \sum_i \sum_j C_{ij}x_{ij}}$$

where:

- C_{ij} is the cost of model j for query i .
- Q_{ij} is the quality of model j for query i .
- x_{ij} is the binary assignment variable.

15 Why Integer Programming?

Model selection is a discrete decision.

A query cannot be partially assigned to two models in the final routing decision.

It must be assigned to one model:

$$\text{Query}_i \rightarrow \text{Model}_j$$

Therefore, binary decision variables are suitable.

16 Optimization Technique 2: Goal Programming

The project has multiple objectives:

- Minimize cost
- Maintain quality
- Reduce latency

These objectives can conflict with one another.

For example, a model with high quality may be expensive, while a cheap model may have lower quality.

Goal Programming is therefore used to find a balance between these competing objectives.

16.1 Goal Programming Objective

The weighted goal programming formulation is:

$$\boxed{\text{Minimize}(w_1d_{\text{cost}} + w_2d_{\text{quality}} + w_3d_{\text{latency}})}$$

where:

- d_{cost} is the deviation from the desired cost.
- d_{quality} is the deviation from the desired quality.
- d_{latency} is the deviation from the desired latency.
- w_1, w_2, w_3 are the importance weights.

For example:

$$w_{\text{quality}} = 0.50$$

$$w_{\text{cost}} = 0.30$$

$$w_{\text{latency}} = 0.20$$

The weights can be changed by the user through the interface.

17 Optional Technique: AHP

Analytic Hierarchy Process (AHP) may be used as a supporting technique.

AHP can rank LLMs based on:

- Quality
- Cost
- Latency

The hierarchy can be represented as:

Model Selection

↓

Quality, Cost, Latency

AHP can generate an overall ranking of models.

However, AHP is optional. The main optimization techniques are Integer Programming and Goal Programming.

18 Optimal Routing

After optimization, every query receives an LLM assignment.

Example:

Table 4: Example optimal routing

Query	Selected Model
Q1	Gemini 2.5 Flash
Q2	DeepSeek V3
Q3	Gemini 2.5 Pro
Q4	Gemini 2.5 Flash
Q5	GPT-5

The actual routing will be determined by the optimization model.

19 Baseline Methods

To prove that the optimization approach provides an improvement, it will be compared with baseline methods.

19.1 Baseline 1: Cheapest Model

Every query is sent to the cheapest available model.

$$\textit{All Queries} \rightarrow \textit{Cheapest Model}$$

19.2 Baseline 2: Strongest Model

Every query is sent to the highest-performing model.

$$\textit{All Queries} \rightarrow \textit{Strongest Model}$$

19.3 Proposed Method

The proposed approach is:

$$\textit{Query} \rightarrow \textit{Capability Prediction} \rightarrow \textit{Optimization} \rightarrow \textit{Selected Model}$$

All approaches will be tested on the same set of queries.

20 Evaluation

The main evaluation metrics are:

20.1 Total Cost

$$\textit{Total Cost} = \sum \textit{Cost}_i$$

20.2 Average Quality

$$\textit{Average Quality} = \frac{\sum \textit{Score}_i}{N}$$

20.3 Average Cost

$$\textit{Average Cost} = \frac{\textit{Total Cost}}{N}$$

20.4 Latency

Latency or execution time will be compared using reliable available measurements.

21 Cost Saving

Cost saving will be calculated by comparing the optimized approach with the baseline.

$$\text{Cost Saving}(\%) = \frac{\text{Cost}_{\text{baseline}} - \text{Cost}_{\text{optimized}}}{\text{Cost}_{\text{baseline}}} \times 100$$

For example, if the baseline costs \$100 and the optimized routing costs \$60:

$$\text{Cost Saving} = \frac{100 - 60}{100} \times 100 = 40\%$$

The final project will use actual experimental results.

22 Quality Comparison

The optimized routing quality will be compared with the strongest-model baseline.

The objective is not necessarily to achieve higher raw accuracy than the strongest model.

Instead, the objective is:

Maintain an acceptable level of quality while significantly reducing cost and potentially reducing latency.

23 Quality-Cost Trade-off

A quality-cost graph will be generated to compare the different approaches.

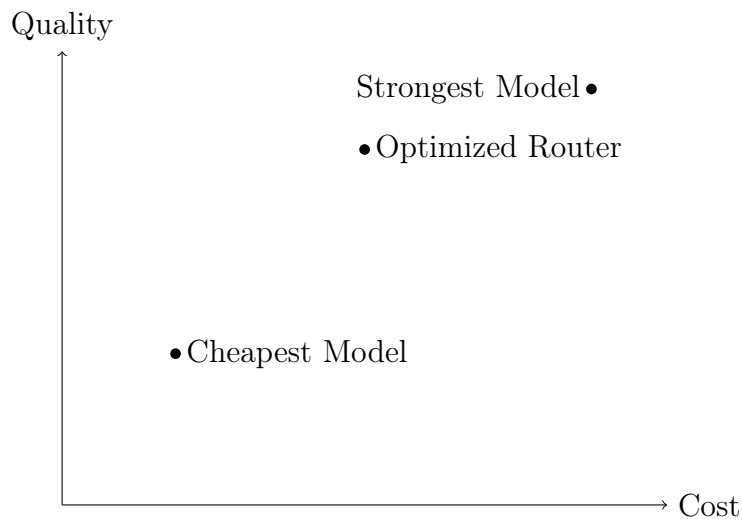


Figure 3: Illustration of quality-cost trade-off

The optimized solution should ideally provide quality close to the strongest model while requiring significantly lower cost.

24 Sensitivity Analysis

Sensitivity analysis will be used to determine how the optimal solution changes when important parameters are changed.

The following parameters can be varied.

24.1 Minimum Quality Requirement

85%, 90%, 95%

24.2 Cost Priority

Different cost weights can be tested.

24.3 Quality Priority

Different quality weights can be tested.

24.4 Latency Priority

Different latency weights can be tested.

The following outcomes will be observed:

- Model selection
- Total cost
- Average quality
- Latency
- Routing distribution

Increasing the quality requirement is expected to reduce the number of feasible low-cost models and may increase the total cost.

25 User Interface

The final application will be implemented using Streamlit.

The interface will contain the following components.

25.1 Dataset Upload

The user can upload a JSON or CSV dataset.

Upload Dataset

[Choose JSON/CSV]

[Load Dataset]

25.2 Statistics Dashboard

The interface will display:

- Number of queries
- Number of models
- Average quality
- Total cost
- Average cost
- Token usage

Charts will include:

- Quality versus model
- Cost versus model
- Token usage versus model
- Quality-cost comparison

25.3 Optimization Settings

The user can specify:

Minimum Quality: 90%

Cost Priority: 30%

Quality Priority: 50%

Latency Priority: 20%

The user can then select:

[Integer Programming]

[Goal Programming]

25.4 Results Dashboard

The results dashboard will display:

- Optimal model for each query
- Total cost
- Average quality
- Average latency
- Cost savings
- Model usage distribution
- Baseline comparison

26 System Architecture

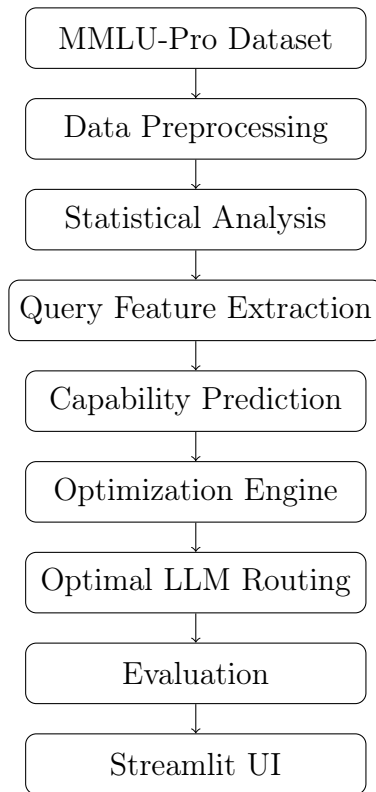


Figure 4: Overall system architecture

27 Technology Stack

27.1 Programming Language

Python

27.2 Data Processing

- Pandas
- NumPy

27.3 Machine Learning

- Scikit-learn

27.4 Optimization

- PuLP

- OR-Tools
- SciPy Optimization

27.5 Visualization

- Matplotlib
- Plotly

27.6 User Interface

Streamlit

28 Project Folder Structure

```
llm-routing-optimization/  
|  
+-- data/  
| +-- raw/  
| | +-- gemini-2.5-flash.json  
| | +-- deepseek-v3.json  
| | +-- gemini-2.5-pro.json  
| | +-- claude-sonnet-4.json  
| | +-- gpt-5.json  
| |  
| +-- processed/  
| +-- llm_routing_dataset.csv  
|  
+-- preprocessing/  
| +-- load_data.py  
| +-- clean_data.py  
| +-- merge_models.py  
|  
+-- analysis/  
| +-- statistics.py  
| +-- visualization.py  
|  
+-- prediction/  
| +-- feature_extraction.py  
| +-- capability_predictor.py  
|
```

```
+-- optimization/
| +-- integer_programming.py
| +-- goal_programming.py
| +-- sensitivity_analysis.py
|
+-- evaluation/
| +-- baseline.py
| +-- metrics.py
| +-- comparison.py
|
+-- app/
| +-- app.py
|
+-- requirements.txt
|
+-- README.md
```

29 Complete End-to-End Procedure

The complete project procedure is:

1. Download LLMRouterBench.
2. Select MMLU-Pro `test_3000`.
3. Select the required LLMs.
4. Extract the JSON records.
5. Combine all model results.
6. Clean and validate the dataset.
7. Perform statistical analysis.
8. Extract query features.
9. Determine required model capability.
10. Train the capability predictor.
11. Create the Integer Programming model.
12. Create the Goal Programming model.

13. Generate optimal routing.
14. Run baseline approaches.
15. Compare cost, quality, and latency.
16. Perform sensitivity analysis.
17. Build the Streamlit UI.
18. Display final results.
19. Draw conclusions.

30 Expected Outcome

The expected outcome is an intelligent routing system that distributes queries across different LLMs according to their requirements.

The expected routing behavior is:

Simple Query

|

v

Low-Cost LLM

Moderate Query

|

v

Medium-Cost LLM

Complex Query

|

v

High-Capability LLM

The system aims to achieve:

- Lower overall cost
- Acceptable response quality
- Reduced or controlled latency
- Better quality-cost trade-off

31 Proof of Optimization Effectiveness

The effectiveness of the optimization will be demonstrated through experimental comparison.

Three approaches will be compared:

1. Cheapest model for every query
2. Strongest model for every query
3. Proposed optimization-based routing

The comparison will use:

- Total cost
- Average quality
- Average latency
- Cost savings
- Model distribution

The optimized approach will be considered successful if it can significantly reduce cost while maintaining an acceptable level of quality.

32 Limitations

The project has the following limitations:

1. MMLU-Pro is a benchmark dataset and may not represent every real-world query.
2. Model pricing can change over time.
3. The project is limited to the models available in the selected benchmark.
4. Capability prediction may occasionally make incorrect predictions.
5. Historical benchmark performance may not perfectly represent real-world performance.
6. Execution-time information may be aggregate or model-level rather than reliable per-query latency.

33 Future Improvements

Future versions of the project can include:

- Real-time LLM API routing
- More LLM providers
- Dynamic model pricing
- Real-time latency monitoring
- Additional benchmark datasets
- Reinforcement-learning-based routing
- RAG-aware routing
- User-specific routing preferences
- Energy-aware model routing
- Automatic model discovery

34 Final Project Summary

The project “Cost-Aware LLM Model Routing Using Optimization” develops a system that selects an appropriate LLM for each query instead of using the same model for every query.

The project uses the MMLU-Pro benchmark from LLMRouterBench to study how different LLMs perform on the same questions and how much each model costs.

Query characteristics and historical model performance are used to estimate the capability required for a query. Integer Programming is then used to assign queries to suitable models while minimizing cost. Goal Programming is used to balance cost, quality, and latency.

The optimized routing strategy is compared with baseline strategies such as always selecting the cheapest model or always selecting the strongest model.

Sensitivity analysis is then performed by changing quality requirements and optimization priorities.

Finally, an interactive Streamlit interface allows users to upload data, view statistics, configure optimization preferences, run the optimization, and visualize the results.

35 Final Project Flow

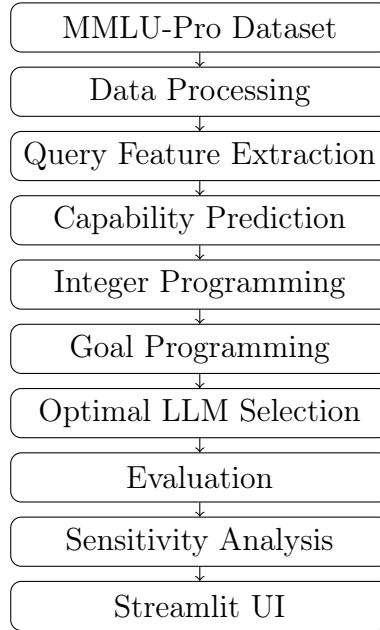


Figure 5: Final project workflow

36 Central Concept

The central concept of the project can be summarized as:

Prediction determines what the query needs. Optimization determines which model should provide it.

The capability prediction component estimates the required model capability from query characteristics and historical model performance. The optimization component then selects the most appropriate LLM while considering cost, quality, and latency.

Therefore, the project combines machine learning with Operations Research techniques to develop a cost-aware and quality-aware LLM routing system.